MINI PROJECT 2: NETWORK PROTOCOL ANALYSIS USING WIRESHARK

Student Name:	Allamsetti Venkata Soumya Sri
Roll / Registration No.:	STU-93A9D6
Batch / Course:	Cyber Security / G17
Academic Platform:	SURE ProEd
Date of Submission:	24-10-2026

1. Introduction to Network Protocol Analysis

A network packet represents the discrete, formatted data unit transmitted across digital networks, encapsulating payload data within structured headers. A network protocol defines the authoritative syntax, semantics, and synchronization rules that communicate between computing systems. In cybersecurity operations and network engineering, packet analysis – often termed packet sniffing or deep packet inspection (DPI) – is the rigorous practice of capturing raw binary frames as they traverse physical or virtual network interfaces and decoding each hierarchical layer from Data Link framing up to Application Layer payloads.

Wireshark is the industry-standard network protocol analyzer. It captures live packet streams across network interface controllers (NICs) via packet capture libraries (libpcap), reconstructing the encapsulation hierarchy defined by the Open Systems Interconnection (OSI) 7-layer reference model and the practical 4-layer TCP/IP suite. For cybersecurity analysts, packet-level visibility provides definitive empirical evidence required for vulnerability assessments, malicious traffic detection, protocol compliance verification, and forensic root-cause analysis during incident investigations.

This project performs an empirical investigation of live multi-protocol network traffic captured within a virtualized Kali Linux environment running on VMware Workstation on a Windows 11 host system. The investigation examines core networking and transport protocols: Domain Name System (DNS), User Datagram Protocol (UDP), Internet Control Message Protocol for IPv6 (ICMPv6) alongside Neighbor Discovery Protocol (NDP), Transmission Control Protocol (TCP), Network Time Protocol (NTP), Dynamic Host Configuration Protocol (DHCP), and Transport Layer Security (TLSv1.3). The analysis correlates low-level packet fields – such as 16-bit transaction IDs, 32-bit sequence/acknowledgement numbers, protocol flags, and cryptographic handshake extensions – with theoretical network models and practical defensive cybersecurity principles.

2. Project Aim and Objectives

Primary Aim: To capture, decode, and evaluate live multi-layer network protocols (DNS, UDP, ICMPv6, TCP, NTP, DHCP, and TLSv1.3) using Wireshark within a controlled virtualization environment, rigorously validating protocol mechanics, state synchronization, addressing semantics, and defensive cybersecurity implications.

Key Objectives: The operational practical objectives include:

- **Interface Capture:** Configure and execute live packet capture on the Kali Linux virtual network interface (eth0) utilizing Wireshark 4.4.7.
- **Traffic Generation:** Generate reproducible network traffic using native command-line utilities, including ICMPv6 reachability tests (`ping -c 6 google.com`) and secure HTTP sessions (`curl -I https://www.microsoft.com`).
- **DNS Resolution:** Dissect DNS query-response exchanges over UDP port 53, validating 16-bit transaction ID matching, recursive resolution flags, and record type mappings.
- **ICMPv6 & NDP:** Analyze ICMPv6 Echo Request/Reply operations and IPv6 Neighbor Discovery Protocol (NDP) solicitations and advertisements, contrasting them with IPv4 ARP.
- **TCP Handshake:** Inspect the full TCP three-way handshake (SYN, SYN-ACK, ACK), verifying sequence/acknowledgement number progression, window scale calculations, and conversation completeness.
- **UDP Services:** Evaluate connectionless UDP transport behavior across auxiliary protocols, specifically NTP Version 4 client timing requests and DHCP IPv4 lease exchanges.
- **TLSv1.3 Security:** Examine TLSv1.3 cryptographic handshake metadata (Client Hello, SNI, ALPN, cipher suites) while delineating metadata visibility from encrypted payload protection.
- **Cybersecurity Mapping:** Construct protocol-specific display filters to isolate traffic streams and map observed protocol behaviors to real-world cybersecurity threat vectors.

3. Theoretical Background and Tools Used

3.1 OSI 7-Layer Reference Model vs. TCP/IP Architecture

Network communication relies on standardized architectural models that partition complex communication tasks into hierarchical layers. The ISO Open Systems Interconnection (OSI) model defines a conceptual 7-layer framework, whereas the TCP/IP model provides the pragmatic 4-layer architecture upon which the global Internet operates. Each layer encapsulates outgoing application data with specific headers containing control, addressing, and sequencing metadata, and decapsulates incoming frames upon reception.

Table 1: Comparison of OSI 7-Layer and TCP/IP Architectural Models and Associated Threat Vectors

OSI Layer	TCP/IP Layer	PDU Unit	Core Operational Role	Cybersecurity Threat Vectors
7. Application	Application	Data	End-user network services (HTTP, DNS, NTP)	Phishing, DNS spoofing, payload injection
6. Presentation	Application	Data	Data syntax, character encoding, encryption (TLS)	Cryptographic downgrade, serialization attacks
5. Session	Application	Data	Session establishment, coordination, and tear-down	Session hijacking, replay attacks, session fixation
4. Transport	Transport	Segment / Datagram	End-to-end reliability (TCP) or datagram speed (UDP)	TCP SYN flood, port scanning, connection reset
3. Network	Internet	Packet	Logical addressing (IPv4/IPv6) and path routing	IP spoofing, ICMP flooding, route poisoning
2. Data Link	Network Access	Frame	Physical MAC delivery and local segment switching	MAC flooding, ARP spoofing, rogue NDP RA
1. Physical	Network Access	Bits	Transmission of raw electrical, optical, or RF signals	Physical wiretapping, rogue hardware, jamming

The conceptual relationship between the seven layers of the OSI model and the four practical layers of the TCP/IP suite is illustrated in Figure 1, emphasizing how data units are transformed during transmission across networked systems.



Figure 1: Architectural Mapping of OSI 7-Layer Reference Model to TCP/IP Protocol Suite.

3.2 Tools and Software Environment

- **Packet Analyzer:** Wireshark 4.4.7: High-performance packet capture and dissection engine utilized for real-time interface monitoring, protocol decoding, and forensic packet filtering.
- **Guest Operating System:** Kali Linux (64-bit): Dedicated cybersecurity Linux distribution hosted as a guest virtual machine running inside VMware Workstation on a Windows 11 host system.
- **Network Utilities:** ping (iputils): Diagnostic utility executed to generate ICMPv6 Echo Requests and stimulate upstream DNS resolution via the glibc resolver.
- **Application Client:** curl (libcurl): Command-line HTTP/HTTPS client utilized with header-only flags (-I) to initiate structured TCP connections and TLSv1.3 handshakes.
- **Network Simulator:** Cisco Packet Tracer: Separate network simulation platform used to illustrate switched and routed topology and inter-subnet packet forwarding (Section 8.4). It is independent of the live Wireshark captures.

4. System Configuration and Wireshark Capture Procedure

4.1 System Configuration and Network Environment

A rigorous protocol analysis requires documenting the precise operating environment, hardware parameters, and network addressing scheme. Table 2 details the exact parameters extracted from the captured packet stream. *(Note on Operational Security: this academic copy retains the authentic MAC and IPv6 addresses captured during the laboratory exercise so that the analysis can be verified; in any publicly distributed version, all host interface identifiers in Tables 2, 4 and 5 and in the figures should be partially masked for OPSEC and privacy protection. Addresses abbreviated with "..." in Table 4 are shown in full in Table 2.)* The Kali Linux virtual machine operated on an IPv6-enabled local segment where the primary network interface (eth0) acquired a globally routable IPv6 address and a link-local address. Crucially, the live VM interface possessed no assigned IPv4 address or default IPv4 gateway during the primary live capture, compelling native utilities (ping, curl) to operate entirely over IPv6. In parallel, a separate IPv4 configuration archive containing DHCP lease exchanges was evaluated to examine legacy bootstrap mechanisms. **Frame numbers are local to each capture file:** Frames 5–46 cited in this report come from the live IPv6 capture on eth0, whereas Frame 49 (DHCP) comes from the separate IPv4 archive.

Table 2: Kali Linux Experimental Environment and Host Addressing Configuration

Host Environment / Hypervisor	Windows 11 Host system executing VMware Workstation Hypervisor
Guest Operating System	Kali Linux Rolling Release (Terminal user context: kali@kali)
Capture Software & Engine	Wireshark Network Analyzer Version 4.4.7 (libpcap capture engine)
Active Capture Interface	eth0 (Ethernet network interface controller, bound in promiscuous mode)
VM Virtual MAC Address	00:0c:29:c5:97:80 (Organizationally Unique Identifier decoded as VMware_c5:97:80)
VM Global IPv6 Address	2409:40e6:435:6928:2ecd:10e4:7158:11a5 (SLAAC / Router Advertised prefix)
VM Link-Local IPv6 Address	fe80::20c:29ff:fec5:9780 (Modified EUI-64 format derived from MAC)
Default Gateway Router	Link-local: fe80::824:45ff:fe96:147f | MAC: 0a:24:45:96:14:7f
Upstream DNS Resolver	2409:40e6:435:6928::91 (Queried over UDP port 53 for recursive resolution)

4.2 Packet Capture Methodology and Display Filtering

The operational capture workflow commenced by launching Wireshark with elevated privileges within the Kali Linux terminal. As illustrated in Figure 2, the Wireshark 4.4.7 startup interface presents active network interfaces alongside real-time sparkline traffic monitors. The primary interface eth0 was selected, and a live promiscuous capture was initialized.

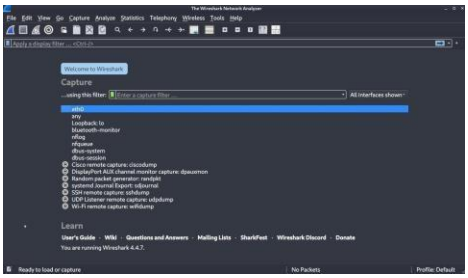


Figure 2: Wireshark 4.4.7 Interface Startup Screen Displaying Interface eth0.

Wireshark utilizes two distinct filtering paradigms: Capture Filters (BPF syntax compiled by libpcap and applied in the kernel capture path) and Display Filters (applied non-destructively to the in-memory capture buffer to isolate specific conversations). Table 3 outlines the key display filters evaluated during this project.

Table 3: Evaluated Wireshark Protocol Display Filters and Practical Applications

Display Filter	Target Protocol	Filter Syntax Rationale	Project Practical Application
`tcp`	TCP Protocol	Isolates all Transmission Control Protocol segments	Applied in Figure 11; the screenshot shows the handshake Frames 5–7 (later TCP segments such as Frame 8 are not shown)
`icmpv6`	ICMPv6 Protocol	Isolates IPv6 control, echo, and neighbor messages	Used to evaluate ping echo packets and NDP solicitations
`dns`	DNS Protocol	Filters queries/responses on standard port 53	Used to inspect A, AAAA, and PTR name resolution frames
`bootp || dhcp`	DHCP Protocol	Isolates bootstrap and DHCP IPv4 exchanges	Evaluated on Frame 49 (IPv4 archive) showing DHCP Request broadcast
`ntp`	NTP Protocol	Filters Network Time Protocol packets on port 123	Evaluated on Frame 39 showing NTPv4 client time sync request

5. DNS and UDP Packet Analysis

5.1 Domain Name Resolution Workflow

To initiate communication with an external domain, the OS resolves the fully qualified domain name (FQDN) into an IP address. Within the Kali Linux terminal, `ping -c 6 google.com` was executed. Although the VM had no IPv4 address (an IPv6-only host, see Section 4.1), the glibc resolver still generated parallel DNS queries over UDP port 53, sent to the resolver over IPv6: an A record query (for IPv4 addresses) and an AAAA record query (for IPv6 addresses). As shown in Figure 3, the host resolver received both resolutions and selected canonical host nw-in-f100.1e100.net at IPv6 address 2404:6800:4013:804::64, the only address family usable on this host, transmitting six ICMPv6 probes with 0% packet loss (avg RTT 69.241 ms).

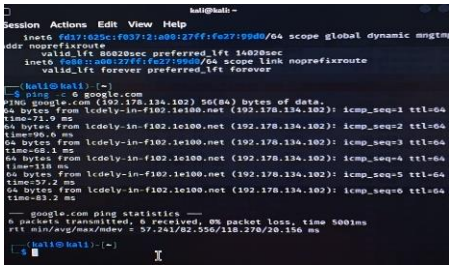


Figure 3: Terminal Output for ping -c 6 google.com.

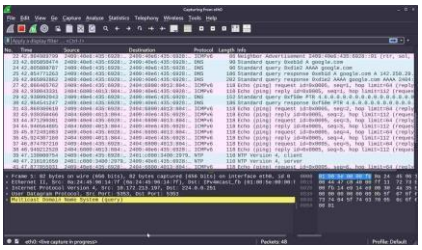


Figure 4: Unfiltered Packet Stream (DNS & ICMPv6).

5.2 Dissection of DNS Query (Frame 23) and Response (Frame 25)

DNS operates as a query-response protocol over UDP port 53. Because UDP is stateless, DNS relies on a pseudo-random 16-bit Transaction ID to correlate answers with requests. Figures 5 and 6 display the dissected protocol trees for the IPv4 resolution exchange: Frame 23 (Query) issues a Type A record request with Transaction ID 0xeb1d, and Frame 25 (Response) echoes ID 0xeb1d, returning 6 IPv4 addresses. Concurrently, a separate AAAA exchange resolved Google's IPv6 address utilized by the ping utility. It is critical to distinguish that while the packet dissections in Figures 5 and 6 specifically evaluate the 32-bit Type A records, the ping utility selected the 128-bit IPv6 address (2404:6800:4013:804::64) via the parallel AAAA lookup. Key parameters for the captured Type A exchange are summarized in Table 4.

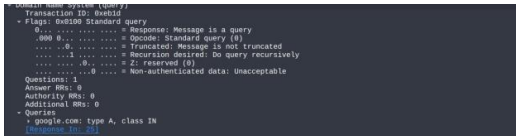


Figure 5: Dissection of Frame 23 (DNS Query) Showing Transaction ID 0xeb1d and Recursion Desired.

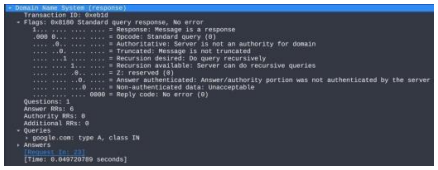


Figure 6: Dissection of Frame 25 (DNS Response) Showing Matching ID 0xeb1d, Flags 0x8180, and 6 Answer Records.

Table 4: Protocol Dissection of DNS Query (Frame 23) vs. DNS Response (Frame 25)

Packet Field / Parameter	DNS Query (Frame 23)	DNS Response (Frame 25)
Source & Destination IP	VM (2409:...:11a5) -> DNS (2409:...::91)	DNS (2409:...::91) -> VM (2409:...:11a5)
UDP Transport Ports	Source Port: 52572 -> Dest Port: 53	Source Port: 53 -> Dest Port: 52572 (Swapped)
Transaction Identifier	0xeb1d (16-bit pseudo-random client token)	0xeb1d (Exact match echoed by resolver)
Protocol Flags	0x0100 (Standard Query, Recursion Desired)	0x8180 (Standard Response, Recursion Available)
Question Section	google.com: type A, class IN	google.com: type A, class IN
Resource Record Counts	Questions: 1, Answers: 0, Authority: 0	Questions: 1, Answers: 6, Authority: 0, Add: 0

5.3 UDP Transport Characteristics in Auxiliary Protocols

The User Datagram Protocol (UDP, RFC 768) provides lightweight, connectionless transport lacking handshake latency, retransmission mechanisms, or window flow control. UDP encapsulates payloads within an 8-byte header (Source Port, Destination Port, Length, Checksum). In DNS, UDP enables near-instantaneous name resolution (Frame 25 arrived in 49.72 ms). UDP's minimal overhead also makes it ideal for Network Time Protocol (NTP port 123) and DHCP (ports 67/68), where message loss is handled via application-level timeouts rather than transport retransmissions.

6. ICMPv6 and Neighbor Discovery Analysis

6.1 ICMPv6 Echo Request and Reply Mechanics

The Internet Control Message Protocol for IPv6 (ICMPv6, RFC 4443) is an integral part of the IPv6 suite, assigned Next Header value 58 in the IPv6 header. Beyond diagnostics, ICMPv6 handles critical control functions, path MTU discovery, and local link-layer address resolution. Figure 7 illustrates the captured ICMPv6 stream containing the alternating Echo Request and Reply sequence alongside interleaved Neighbor Discovery Protocol packets.

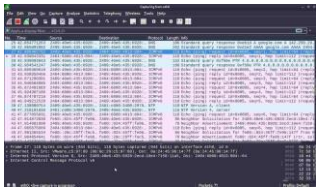


Figure 7: Wireshark Packet List Showing ICMPv6 Echo Requests/Replies and Neighbor Discovery Frames.

Figure 8 and Figure 9 present the protocol dissections for Frame 27, representing the initial ICMPv6 Echo Request generated by the ping utility, with observed parameters detailed in Table 5.



Figure 8: Frame 27 ICMPv6 Header Dissection Showing Type 128 (Echo Request), Identifier 0x0005, and Sequence 1.

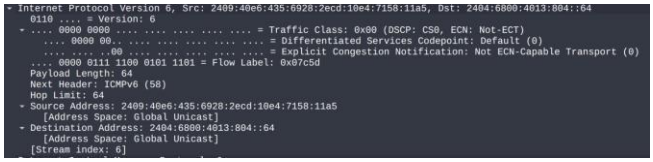


Figure 9: Frame 27 IPv6 Network Header Showing Next Header 58 (ICMPv6), Hop Limit 64, and Payload Length 64.

Table 5: ICMPv6 Echo Request/Reply Sequence and Hop Progression Metrics

Source IPv6 -> Destination IPv6	2409:40e6:435:6928:2ecd:10e4:7158:11a5 (VM) -> 2404:6800:4013:804::64 (google.com)
Source MAC -> Destination MAC	00:0c:29:c5:97:80 (VM NIC) -> 0a:24:45:96:14:7f (Default Gateway Router)
Captured Echo Frame Pairs	Sequence 1 (27/28), Seq 2 (31/32), Seq 3 (33/34), Seq 4 (35/36), Seq 5 (37/38), Seq 6 (41/44)
ICMPv6 Identifier & Sequence	Identifier: 0x0005 constant across all probes; Sequence increments monotonically (1 to 6)
Hop Limit Progression	Echo Requests: Outbound Hop Limit = 64 (VM default); Echo Replies: Observed Hop Limit = 112 (received from target)
Empirical Round-Trip Time (Seq 1)	Frame 28 timestamp (42.930843 s) - Frame 27 timestamp (42.866465 s) = 64.38 ms

6.2 IPv6 Neighbor Discovery Protocol (NDP) vs. IPv4 ARP

A fundamental distinction between IPv4 and IPv6 lies in link-layer address resolution. While IPv4 relies on Address Resolution Protocol (ARP, RFC 826) utilizing unauthenticated broadcast frames, IPv6 replaces ARP with Neighbor Discovery Protocol (NDP, RFC 4861). NDP operates at the Network Layer using dedicated ICMPv6 message types transmitted via solicited-node multicast addresses, eliminating broadcast noise on local network segments.

In the captured trace (Figure 7), Frames 42, 43, 45, and 46 demonstrate live NDP cache maintenance. In Frame 42, the default gateway router (fe80::824:45ff:fe96:147f) emitted an ICMPv6 Neighbor Solicitation (Type 135) to verify the VM's link-layer reachability. In Frame 43, the Kali VM replied with an ICMPv6 Neighbor Advertisement (Type 136) providing its physical MAC address (00:0c:29:c5:97:80). In Frames 45 and 46, the VM queried the router, which responded with an advertisement carrying the rtr (Router) and sol (Solicited) flags, ensuring bidirectional frame forwarding without broadcast overhead.

7. TCP Three-Way Handshake Analysis

7.1 TCP Transport Architecture and Connection Establishment

The Transmission Control Protocol (TCP, RFC 9293) provides a reliable, connection-oriented, full-duplex byte-stream transport service at Layer 4 of the OSI model. Unlike UDP, TCP guarantees delivery order, performs congestion control, and handles retransmission of lost segments. Before upper-layer application protocols (such as HTTP or TLS) can transmit data, TCP must establish a synchronized communication session using the canonical three-way handshake: SYN -> SYN-ACK -> ACK.

To stimulate genuine TCP transport traffic, the terminal command `curl -I https://www.microsoft.com` was executed. As illustrated in Figure 10, the Microsoft edge server responded with HTTP/2 200 OK via the AkamaiGHost content delivery network, confirming successful TCP connection establishment and secure TLS negotiation over destination TCP port 443.

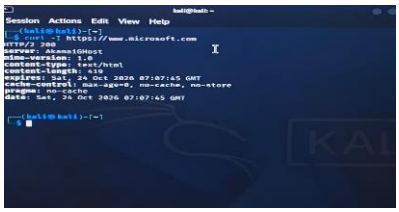


Figure 10: Terminal Output of curl -I Returning HTTP/2 200 OK from Edge Server AkamaiGHost.

Applying the Wireshark display filter `tcp` isolates the TCP traffic from background (non-TCP) traffic. Figure 11 shows the three consecutive handshake segments: Frame 5 [SYN], Frame 6 [SYN, ACK], and Frame 7 [ACK]. (The screenshot is limited to these three frames; a plain `tcp` filter would also list later segments of the same stream, such as the TLS Client Hello in Frame 8.)



Figure 11: Display Filter 'tcp' Showing Handshake Frames 5 [SYN], 6 [SYN, ACK], and 7 [ACK].

7.2 Dissection of Handshake Frames 5, 6, and 7

Figures 12, 13, and 14 show the exact protocol tree dissections for the three handshake frames, detailing sequence numbers, acknowledgement numbers, TCP flags, and negotiated TCP options.

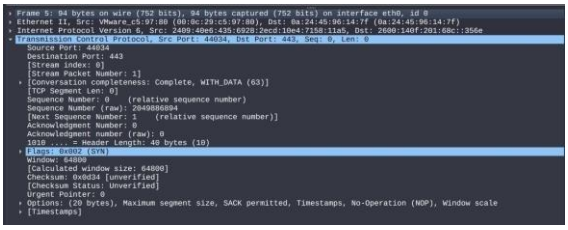


Figure 12: Frame 5 Dissection (SYN from VM): Ephemeral Port 44034 -> Port 443, Raw Seq 2049886894, Flags 0x002.

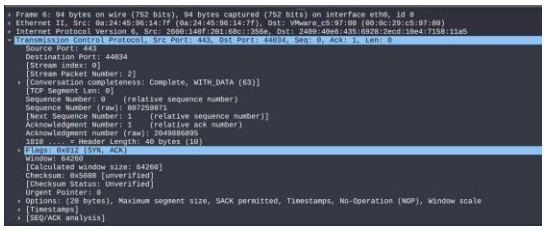


Figure 13: Frame 6 Dissection (SYN-ACK from Server): Raw Seq 807259871, Raw Ack 2049886895, Flags 0x012.

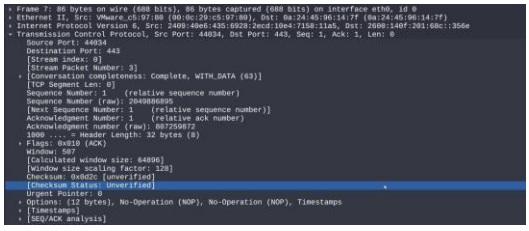


Figure 14: Frame 7 Dissection (ACK from VM): Raw Seq 2049886895, Raw Ack 807259872, Flags 0x010.

7.3 TCP Handshake Parameter Verification and Mathematical Proof

The mathematical synchronization of TCP sequence and acknowledgement numbers is detailed in Table 6. The exchange begins in Frame 5 where the client chooses a pseudo-random Initial Sequence Number (ISN_c = 2049886894). In Frame 6, the server acknowledges the SYN by incrementing ISN_c by 1 (Ack = 2049886895) and presents its own random ISN_s (807259871). In Frame 7, the client acknowledges the SYN-ACK by incrementing ISN_s by 1 (Ack = 807259872). Wireshark marks this stream Conversation completeness: Complete, WITH_DATA.

Table 6: Mathematical Synchronization of TCP Handshake Parameters (Frames 5–7)

Frame	Direction (Ports)	TCP Control Flags	Raw Sequence No.	Raw Ack No.	Window / Options
Frame 5	VM -> Server (44034 -> 443)	0x002 (SYN)	2049886894 (Rel: 0)	0 (Not set)	Win: 64800, MSS: 1440, WS: 128
Frame 6	Server -> VM (443 -> 44034)	0x012 (SYN, ACK)	807259871 (Rel: 0)	2049886895 (ISN_c + 1)	Win: 64260, MSS: 1440, WS: 128
Frame 7	VM -> Server (44034 -> 443)	0x010 (ACK)	2049886895 (Rel: 1)	807259872 (ISN_s + 1)	Win: 507 (x128 = 64896 bytes)

8. Network Services, TLSv1.3 Security, and Infrastructure Simulation

8.1 Network Time Protocol (NTP) Dissection (Frame 39)

Accurate system clock synchronization is a paramount operational requirement in modern networks, particularly for correlating security events, intrusion detection logs, and digital forensics evidence. Figure 15 dissects Frame 39, displaying an outbound NTP Version 4 client request emitted over IPv6 via UDP port 37534 to standard NTP destination port 123.

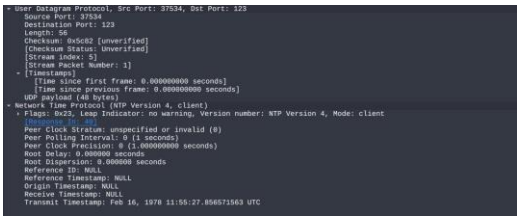


Figure 15: Dissection of Frame 39 (NTP Version 4 Client) Showing UDP Port 123 and 48-Byte Payload.

The packet carries NTP flags 0x23 (Leap Indicator 0: no warning; Version 4; Mode 3: Client). The UDP payload length is exactly 48 bytes, the standard NTP header size. In outbound client Mode 3 requests, RFC 5905 defines that the client reports its own Peer Clock Stratum as 0 (unspecified) because it is requesting time rather than serving as a synchronized reference. Wireshark automatically tracks the exchange with metadata '[Response In: 40]', identifying a corresponding response in Frame 40; the empirical analysis here is strictly confined to the verified client fields in Frame 39 without asserting uninspected response payload data.

8.2 Dynamic Host Configuration Protocol (DHCP) Dissection (Frame 49)

Figure 16 examines Frame 49 from the supplementary IPv4 evidence archive (a separate capture file from the live eth0 capture, so its frame numbering is independent), capturing a DHCP Request emitted by client MAC 00:0c:29:c5:97:80 during dynamic IPv4 bootstrap. The exchange occurs over UDP source port 68 to destination port 67, using IPv4 broadcast destination 255.255.255.255.

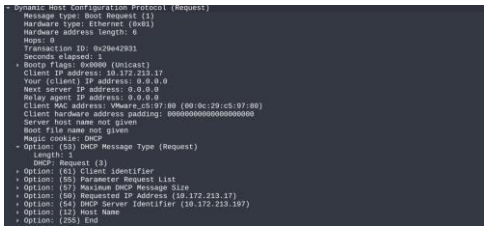


Figure 16: Dissection of Frame 49 (DHCP Request) Displaying Transaction ID 0x29e42931 and Option 50.

The message contains Transaction ID 0x29e42931 and incorporates DHCP Option 53 (Message Type: Request), Option 50 (Requested IP Address: 10.172.213.17), and Option 54 (Server Identifier: 10.172.213.197). This evidence confirms that an isolated DHCP Request broadcast was observed on the wire. Critically, because only Frame 49 is present in this capture snippet, this observation documents the transmission of a DHCP Request; it does not demonstrate or imply that the entire four-way DORA sequence or final DHCP ACK lease binding was successfully completed.

8.3 Transport Layer Security (TLSv1.3) Handshake Analysis

Following successful completion of the TCP three-way handshake in Frames 5-7, the client initiates secure session negotiation via Transport Layer Security (TLSv1.3, RFC 8446). Figure 17 dissects Frame 8, representing the TLS Client Hello message transmitted over TCP port 443 with a total length of 1,652 bytes on the wire.

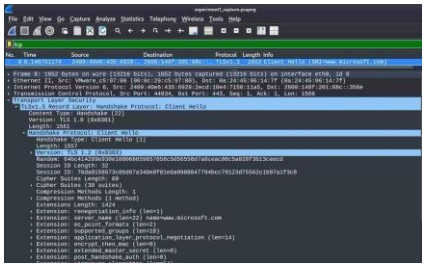


Figure 17: Dissection of Frame 8 (TLSv1.3 Client Hello) Showing SNI (www.microsoft.com) and 30 Cipher Suites.

Cryptographic Metadata: Key cryptographic parameters exposed within the unencrypted Client Hello metadata include:

- **Virtual Hosting: Server Name Indication (SNI):** The extension explicitly indicates 'www.microsoft.com', allowing upstream multi-tenant proxy servers and CDN edges to direct the handshake to the appropriate virtual host certificate.
- **Cipher & Key Exchange:** The Client Hello advertises supported cipher suites and key-exchange parameters used during secure session negotiation.
- **ALPN Extension: Application-Layer Protocol Negotiation (ALPN):** Advertises support for HTTP/2 ('h2') and HTTP/1.1 ('http/1.1').

Cybersecurity Principle: Security Distinction: While TLSv1.3 provides robust payload confidentiality and integrity, the initial handshake metadata (SNI, supported extensions, cipher lists) is visible to passive network sniffers in a standard Client Hello, unless Encrypted Client Hello (ECH) is in use, which can conceal the SNI. Once key agreement concludes, all subsequent application payloads are encrypted, preventing passive inspection without endpoint TLS termination or pre-shared decryption keys.

8.4 Cisco Packet Tracer Topology and Routing Context (Separate Simulation)

To illustrate how lower-layer framing, switching, and routing support higher-layer protocol interactions such as those analyzed in Wireshark, a separate network was modeled in Cisco Packet Tracer. **This simulation is independent of the Wireshark capture:** the live traffic analyzed in Sections 5–8.3 travelled from the Kali VM through the VMware virtual network and the real upstream network, not through this lab topology. Figure 18 illustrates the multi-subnet topology consisting of PCs, Layer 2 switches, and edge routers interconnected via serial links (networks 10.10.10.0/24, 20.20.20.0/24, and a serial WAN 30.30.30.0/24; for a two-router point-to-point link a /30 prefix would be the more conventional choice).

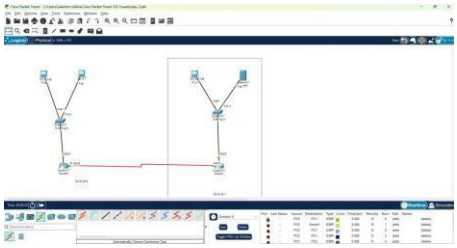


Figure 18: Cisco Packet Tracer Routed Infrastructure.



Figure 19: End-to-End HTTP Delivery ('soumya sri') in Packet Tracer.

Static routing and NAT were configured across Router 7 (Serial 2/0: 30.30.30.1) and Router 6 (Serial 2/0: 30.30.30.2). In Figure 19, an HTTP request from PC0 across the routed topology successfully rendered the customized web page displaying 'soumya sri', showing that the simulated routed path forwarded the HTTP request and response end to end. Packet Tracer is a simulator; this result illustrates routing concepts and does not by itself validate the live Wireshark captures or demonstrate behavior at every OSI layer.

9. Results, Observations, Limitations, and Cybersecurity Relevance

9.1 Experimental Results Summary

Table 7 provides a consolidated technical summary of all seven network protocols captured and decoded during the experimental practical, documenting the specific protocol layer, transport mechanisms, frame identifiers, and empirical findings. Frame numbers refer to the live eth0 capture unless marked “IPv4 archive”.

Table 7: Consolidated Multi-Protocol Empirical Findings and Layer Mapping

Protocol	Layer	Transport / Port	Frame Range	Empirical Observation / Validation
DNS	Application	UDP port 53	Frames 23-26, 29-30	Matched ID 0xeb1d; Type A returned 6 IPv4 records in 49.7 ms; parallel AAAA resolved IPv6 nw-in-f100
UDP	Transport	Ports 53, 123, 67/68	Frames 23, 39 (live); 49 (IPv4 archive)	Stateless, low-overhead transport for queries, time sync, and DHCP leases
ICMPv6	Network	IPv6 Next Header 58	Frames 27-28, 31-38, 41, 44	Verified 6/6 Echo Request/Replies (0% loss); average RTT 69.2 ms
NDP	Network / Link	ICMPv6 (135, 136)	Frames 42-43, 45-46	Link-local MAC resolution (NS/NA) without broadcast; replaced IPv4 ARP
TCP	Transport	Ports 44034 <-> 443	Frames 5-7	Full SYN -> SYN-ACK -> ACK handshake; verified sequence math and completeness
NTP	Application	UDP port 123	Frame 39	NTPv4 client request (flags 0x23, 48 bytes, Stratum 0); Wireshark tracks [Response In: 40]
DHCP	Application	UDP ports 68 -> 67	Frame 49 (IPv4 archive)	Isolated IPv4 DHCP Request observed (ID 0x29e42931, IP 10.172.213.17); full DORA not implied
TLSv1.3	Security / App	TCP port 443	Frame 8	Client Hello with SNI (www.microsoft.com), 30 ciphers and key shares; payload encrypted

9.2 Experimental Limitations

- **Network Protocol Split: Environment Heterogeneity:** The live capture on interface eth0 was exclusively IPv6-focused, whereas DHCP traffic was captured in an auxiliary IPv4 archive with its own frame numbering.
- **Packet Truncation: Collapsed Detail Panes:** In the raw DNS capture, the Answer section was collapsed, truncating trailing IPv4 octets in the packet list summary.
- **Cryptographic Boundary: Payload Opacity:** In the TLSv1.3 capture, security encryption legitimately prevented application-layer HTTP/2 payload inspection from packet captures alone.
- **Simulation Scope:** The Cisco Packet Tracer topology is a separate simulation and was not captured with Wireshark; it supports the routing discussion only.

9.3 Cybersecurity Threat Relevance and Defensive Countermeasures

- **DNS Threat Vectors:** Unauthenticated DNS is susceptible to DNS Cache Poisoning (Kaminsky attacks) and DNS Spoofing, redirecting users to malicious infrastructure. Mitigation requires deploying DNSSEC (cryptographic signing of resource records) and DNS-over-HTTPS (DoH).
- **TCP Exploits:** Attackers exploit TCP’s connection state machine via TCP SYN Flood Denial-of-Service attacks (exhausting server connection backlogs) and TCP SYN stealth scanning (reconnaissance without completing handshakes). Defenses include SYN Cookies and stateful firewall inspection.
- **IPv6 Link Exploits:** Unlike ARP, IPv6 NDP relies on ICMPv6 multicast, but remains vulnerable to Rogue Router Advertisements (man-in-the-middle default gateways) and NDP Cache Poisoning. Mitigation requires implementing RA Guard and Secure Neighbor Discovery (SEND, RFC 3971).
- **Access Layer Threats:** Rogue DHCP servers can assign malicious DNS and gateway settings, while DHCP Starvation exhausts address pools. Defenses mandate DHCP Snooping. Cleartext protocols (HTTP/Telnet) permit credential harvesting, underscoring mandatory TLS 1.3 encryption.

10. Conclusion and References

10.1 Conclusion

This project accomplished a comprehensive, multi-layer protocol analysis using Wireshark 4.4.7 within a virtualized Kali Linux environment, successfully correlating theoretical network models with genuine packet captures. The investigation provided rigorous empirical verification across all primary communication layers:

- **Application Layer:** Validated DNS recursive resolution, matching 16-bit transaction IDs (0xeb1d) over UDP port 53, dissecting NTP Version 4 timing requests, and examining a DHCP Request from the supplementary IPv4 archive.
- **Transport Layer:** Contrasted the lightweight, connectionless operation of UDP against the synchronized, reliable state machine of TCP, verifying the full three-way handshake (SYN, SYN-ACK, ACK) and window scale factors over port 443.
- **Network Layer:** Analyzed IPv6 global and link-local addressing, demonstrating ICMPv6 Echo reachability (0% packet loss) and evaluating Neighbor Discovery Protocol (NDP) mechanics as the modern, multicast-based successor to IPv4 ARP.
- **Security Layer:** Dissected TLSv1.3 Client Hello metadata (SNI, ALPN, cipher suites), establishing the critical security boundary between unencrypted negotiation headers and cryptographically secure application payloads.

Ultimately, deep packet inspection represents an indispensable competency for cybersecurity professionals. The ability to isolate specific conversational flows using display filters, decode raw binary headers, and verify protocol state transitions enables analysts to identify network misconfigurations, detect unauthorized reconnaissance, and remediate advanced cyber threats effectively.

10.2 Academic and RFC References

[1] Wireshark Foundation, “Wireshark User’s Guide,” Version 4.4. [Online]. Available: https://www.wireshark.org/docs/wsug_html_chunked/ (accessed Oct. 24, 2026).

[2] Wireshark Foundation, “Wireshark Display Filter Reference.” [Online]. Available: https://www.wireshark.org/docs/dfref/ (accessed Oct. 24, 2026).

[3] P. Mockapetris, “Domain Names - Implementation and Specification,” RFC 1035, STD 13, IETF, Nov. 1987. doi: 10.17487/RFC1035.

[4] A. Conta, S. Deering, and M. Gupta, “Internet Control Message Protocol (ICMPv6) for the IPv6 Specification,” RFC 4443, Mar. 2006.

[5] T. Narten, E. Nordmark, W. Simpson, and H. Soliman, “Neighbor Discovery for IP version 6 (IPv6),” RFC 4861, Sep. 2007.

[6] W. Eddy, “Transmission Control Protocol (TCP) Specification,” RFC 9293, STD 7, IETF, Aug. 2022. doi: 10.17487/RFC9293.

[7] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” RFC 8446, IETF, Aug. 2018. doi: 10.17487/RFC8446.

[8] R. Droms, “Dynamic Host Configuration Protocol,” RFC 2131, IETF, Mar. 1997. doi: 10.17487/RFC2131.

[9] D. Mills, J. Martin, J. Burbank, and W. Kasch, “Network Time Protocol Version 4: Protocol and Algorithms Specification,” RFC 5905, Jun. 2010.

[10] Cisco Networking Academy, “Introduction to Networks (CCNA) course materials,” Cisco Systems. [Online]. Available: https://www.netacad.com (accessed Oct. 24, 2026).